Sorting Performance Analyzer

Complexity Analysis & Performance Report

Data Structures | ETCCDS202 | Unit 3

School of Engineering & Technology, K.R. Mangalam University

Student: Kinshuk | Roll No.: 2501730171

Program: B.Tech CSE (AI/ML) | Semester: 2

April 2025

1. Algorithms Implemented

1.1 Insertion Sort

Insertion Sort builds the sorted array one element at a time. For each element, it shifts larger elements one position to the right and inserts the current element in its correct position. It is **stable** (equal elements keep original order) and **in-place** (no extra memory needed).

Best case: O(n) — when input is already sorted, inner loop never executes. **Average/Worst:** $O(n^2)$ — each element may shift past all previous elements.

1.2 Merge Sort

Merge Sort divides the array in half recursively until single elements remain, then merges pairs back in sorted order. The merge step compares elements from left and right halves using <= to preserve stability. It is **stable** and **out-of-place** (requires O(n) auxiliary space for merge buffers).

All cases: O(n log n) — the divide step is O(log n) levels deep and each merge level processes O(n) elements total.

1.3 Quick Sort

Quick Sort selects a pivot (last element — Lomuto scheme), partitions the array so all elements <= pivot are to its left and all > pivot are to its right, then recursively sorts both partitions. It is **unstable** and **in-place**.

Average: O(n log n) — pivot tends to split array roughly in half on random data. **Worst case:** $O(n^2)$ — occurs when pivot is always the smallest or largest element (e.g., already sorted array with last-element pivot), causing partitions of size n-1 and 0 every time.

2. Experimental Results — All 27 Measurements

All times measured in milliseconds (ms). Same random seed (42) used for reproducibility. Each algorithm received an identical copy of the data. Sizes tested: 1,000 / 5,000 / 10,000 integers.

Size	Input Type	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
1000	Random	13.81	1.23	0.86
1000	Sorted	0.09	0.85	39.76
1000	Reverse	44.30	0.87	22.81
5000	Random	390.33	7.38	4.88
5000	Sorted	0.35	5.20	885.45
5000	Reverse	690.62	4.80	618.28
10000	Random	1469.04	16.13	10.66
10000	Sorted	0.75	10.96	3367.93
10000	Reverse	2944.14	10.58	2312.35

Table 1: Execution times for all 27 experiments (3 algorithms × 3 sizes × 3 input types).

3. Analysis by Input Type

3.1 Random Input

On random data, both Merge Sort and Quick Sort are dramatically faster than Insertion Sort. At n=10,000, Insertion Sort took 1,469 ms while Merge Sort took only 16 ms and Quick Sort took 11 ms — roughly a 100x difference. This confirms the $O(n^2)$ vs $O(n \log n)$ gap: for n=10,000, n^2=100,000,000 while $n \log n \approx 133{,}000$ — about 750 times fewer operations in theory. Quick Sort slightly outperforms Merge Sort on random data due to better cache behavior and no memory allocation overhead.

3.2 Already Sorted Input

This is Insertion Sort's best case — it makes only $O(n)$ comparisons with zero swaps, running in under 1 ms even for n=10,000. Merge Sort performs consistently at $O(n \log n)$ regardless of input order. Quick Sort, however, degrades severely: with last-element pivot on sorted data, every partition produces an empty left side and an n-1 right side. This $O(n^2)$ behaviour shows clearly — at n=10,000, Quick Sort took 3,368 ms, over 300x slower than on random data.

3.3 Reverse-Sorted Input

Reverse-sorted input is the absolute worst case for Insertion Sort — every new element must shift past all previously sorted elements, giving $O(n^2)$ comparisons and swaps. At n=10,000 it took 2,944 ms. Quick Sort also suffers here: the last element is always the smallest, making every partition maximally unbalanced — 2,312 ms at n=10,000. Merge Sort remains unaffected at ~11 ms, consistent with its

guaranteed O(n log n) performance on all input types.

4. Theoretical Complexity vs Observed Results

The results match theory very precisely. The table below shows that when input size grows from 1,000 to 10,000 (10x), the observed time for $O(n^2)$ algorithms grows roughly 100x, and for O(n log n) algorithms roughly 13x — exactly matching the mathematical ratios.

Algorithm	Input	Time at n=1000	Time at n=10000	Ratio (observed)	Expected ratio
Insertion Sort	Random	13.81 ms	1469.04 ms	~106x	~100x (n^2)
Insertion Sort	Reverse	44.30 ms	2944.14 ms	~66x	~100x (n^2)
Merge Sort	Random	1.23 ms	16.13 ms	~13x	~13x (n log n)
Merge Sort	Sorted	0.85 ms	10.96 ms	~13x	~13x (n log n)
Quick Sort	Random	0.86 ms	10.66 ms	~12x	~13x (n log n)
Quick Sort	Sorted	39.76 ms	3367.93 ms	~85x	~100x (n^2)

The close match between observed and expected ratios validates both the implementations and the theoretical analysis.

5. Quick Sort — Worst Case Behaviour

With Lomuto partition using the **last element as pivot**, Quick Sort hits its worst case whenever the input is already sorted or reverse-sorted. In both cases, the pivot is always the extreme element, producing partitions of sizes (0, n-1) at every level. This forces n recursive calls instead of log n, giving $O(n^2)$ time.

```
Sorted input, n=10000: Merge Sort → 10.96 ms (O(n log n), unaffected) Quick Sort
→ 3367.93 ms (O(n²) worst case — 307x slower!) Reverse input, n=10000: Merge
Sort → 10.58 ms Quick Sort → 2312.35 ms (O(n²) worst case — 218x slower!)
```

Warning: Never use last-element pivot Quick Sort on pre-sorted data in production. Use randomised pivot, median-of-three pivot, or Python's built-in Timsort instead.

Mitigations: (1) Randomise the pivot before each partition. (2) Use median-of-three (first, middle, last) as pivot. (3) Switch to Insertion Sort for small sub-arrays (n < 10–20). These changes keep Quick Sort at O(n log n) in practice.

6. Stability and Memory Properties

Algorithm	Stable?	In-place?	Extra Space	Why
Insertion Sort	Yes	Yes	O(1)	Swaps adjacent elements; equal elements never cross
Merge Sort	Yes	No	O(n)	Creates new left/right sub-arrays during merge

Quick Sort	No	Yes	O(log n)*	Swa	s can move equal elements past each other

** Quick Sort uses O(log n) stack space for recursion on average (O(n) in worst case when already sorted).*

What does stable mean?

A sort is **stable** if two elements with equal values appear in the same relative order in the output as in the input. For example, sorting students by marks: if Alice and Bob both scored 75, a stable sort keeps Alice before Bob if she appeared first in the original list. Insertion Sort and Merge Sort are stable; Quick Sort (typical implementation) is not.

What does in-place mean?

An **in-place** algorithm sorts the data within the original array using only a constant amount of extra memory (O(1) auxiliary space, not counting the call stack). Insertion Sort and Quick Sort are in-place. Merge Sort requires an extra O(n) array to hold merged results, making it out-of-place.

7. Practical Recommendations

Scenario	Best Choice	Reason
Small array (n < 50)	Insertion Sort	Low overhead; fast in practice for tiny n
Large random data	Quick Sort	Fastest average case; good cache performance
Nearly sorted data	Insertion Sort	O(n) best case; very few comparisons needed
Sorted or reverse-sorted data	Merge Sort	Guaranteed O(n log n); Quick Sort degrades
Stability required	Merge Sort	Only O(n log n) stable sort among the three
Memory constrained	Quick Sort	In-place; Merge Sort needs O(n) extra space
General purpose (production)	Timsort	Python's built-in; hybrid Merge+Insertion

8. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. [Chapter 2 — Sorting; Chapter 7 — Quick Sort; Chapter 8 — Merge Sort]

2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley.

3. GeeksforGeeks — Quick Sort, Merge Sort, Insertion Sort articles. https://www.geeksforgeeks.org/

4. Python Official Documentation — sys.setrecursionlimit, time.perf_counter. https://docs.python.org/3/

5. Course notes: Data Structures (ETCCDS202), K.R. Mangalam University, Unit 3.